

```

import sys

import tkinter as tk

from tkinter import ttk, messagebox

from config import (create_app_directories, APP_DATA_DIR, APP_NAME, APP_VERSION)

from config import AppConfig

from core.database.db_manager import DatabaseManager

from core.url import URLManager

from core.cloud.auth_manager import GoogleAuthManager

from core.cloud.drive_resolver import DriveResolver

from core.cloud.drive_scanner import DriveScanner

from core.sync.sync_manager import SyncManager


class NexoraUI(tk.Tk):

    """Tkinter Test UI Class for Live Sync Monitoring"""

    def __init__(self, db_manager: DatabaseManager, active_account_id: int):

        super().__init__()

        self.title(f"{APP_NAME} v{APP_VERSION} - Sync Controller")

        self.geometry("540x400")

        self.resizable(False, False)

        self.db = db_manager

        self.sync_mgr = SyncManager()

        self.default_account_id = active_account_id

        self._init_ui()

        self._start_status_polling()

    def _init_ui(self):

        # 1. Inputs Section

```

```

input_frame = ttk.LabelFrame(self, text=" Drive & Job Configuration ")

input_frame.pack(fill="x", padx=15, pady=10)

ttk.Label(input_frame, text="Account ID:").grid(row=0, column=0, padx=10, pady=8,
sticky="w")

self.ent_account_id = ttk.Entry(input_frame, width=12)

self.ent_account_id.insert(0, str(self.default_account_id or 1))

self.ent_account_id.grid(row=0, column=1, padx=10, pady=8)

ttk.Label(input_frame, text="Drive ID:").grid(row=0, column=2, padx=10, pady=8,
sticky="w")

self.ent_drive_id = ttk.Entry(input_frame, width=20)

self.ent_drive_id.insert(0, "root")

self.ent_drive_id.grid(row=0, column=3, padx=10, pady=8)

ttk.Label(input_frame, text="Type:").grid(row=1, column=0, padx=10, pady=8, sticky="w")

self.combo_type = ttk.Combobox(input_frame, values=["my_drive", "shared_drive"],
width=10, state="readonly")

self.combo_type.set("my_drive")

self.combo_type.grid(row=1, column=1, padx=10, pady=8)

# 2. Controls Section

btn_frame = ttk.LabelFrame(self, text=" Sync Thread Controls ")

btn_frame.pack(fill="x", padx=15, pady=5)

ttk.Button(btn_frame, text="▶ Start / Resume", command=self.start_sync).grid(row=0,
column=0, padx=8, pady=10)

ttk.Button(btn_frame, text="⏏ Pause", command=self.pause_sync).grid(row=0, column=1,
padx=8, pady=10)

ttk.Button(btn_frame, text="▶ Resume", command=self.resume_sync).grid(row=0,
column=2, padx=8, pady=10)

ttk.Button(btn_frame, text="⏹ Stop", command=self.stop_sync).grid(row=0, column=3,
padx=8, pady=10)

```

3. Live Dashboard Section

```
status_frame = ttk.LabelFrame(self, text=" Live Database Progress ")
status_frame.pack(fill="both", expand=True, padx=15, pady=10)

self.lbl_status = ttk.Label(status_frame, text="Status: IDLE", font=("Helvetica", 11, "bold"))
self.lbl_status.pack(anchor="w", padx=15, pady=6)

self.lbl_files = ttk.Label(status_frame, text="Scanned Files: 0", font=("Helvetica", 10))
self.lbl_files.pack(anchor="w", padx=15, pady=2)

self.lbl_folders = ttk.Label(status_frame, text="Scanned Folders: 0", font=("Helvetica", 10))
self.lbl_folders.pack(anchor="w", padx=15, pady=2)

self.lbl_job_id = ttk.Label(status_frame, text="Active Job ID: None", font=("Helvetica", 9),
foreground="gray")
self.lbl_job_id.pack(anchor="w", padx=15, pady=6)

def start_sync(self):
    try:
        acc_id = int(self.ent_account_id.get().strip())
        drive_id = self.ent_drive_id.get().strip()
        drive_type = self.combo_type.get().strip()
        self.sync_mgr.start_sync(
            account_id=acc_id,          drive_id=drive_id,          drive_type=drive_type,
            drive_name="Shared Drive" if drive_type == "shared_drive" else "My Drive"      )
    except ValueError:
        messagebox.showerror("Error", "Account ID သည် ကိန်းဂဏန်း (Number) ဖြစ်ရပါမည်။")

def pause_sync(self):
    self.sync_mgr.pause_sync(self.ent_drive_id.get().strip())

def resume_sync(self):
    self.sync_mgr.resume_sync(self.ent_drive_id.get().strip())
```

```

def stop_sync(self):
    self.sync_mgr.stop_sync(self.ent_drive_id.get().strip())

def _start_status_polling(self):
    drive_id = self.ent_drive_id.get().strip()

    if drive_id:
        job = self.db.fetch_one(
            "SELECT id, status, scanned_files, scanned_folders FROM sync_jobs WHERE drive_id = ?
ORDER BY id DESC LIMIT 1",
            (drive_id,)
        )

        if job:
            colors = {
                "running": "green",
                "paused": "orange",
                "cancelled": "red",
                "completed": "blue",
                "failed": "red"
            }

            self.lbl_status.config(text=f"Status: {job['status'].upper()}",
foreground=colors.get(job["status"], "black"))

            self.lbl_files.config(text=f"Scanned Files: {job['scanned_files']:,}")

            self.lbl_folders.config(text=f"Scanned Folders: {job['scanned_folders']:,}")

            self.lbl_job_id.config(text=f"Active Job ID: #{job['id']}")

            self.after(500, self._start_status_polling)

def main():
    print("🚀 Initializing Nexora Engine...\n")

```

1. Folders ဖန်တီးခြင်း

```
create_app_directories()
```

```
print("📁 System directories initialized.")
```

2. Database Initialization

```
db = DatabaseManager()
```

```
print("📄 SQLite Database initialized successfully.")
```

3. Auth Manager & Credentials Test

```
auth_mgr = GoogleAuthManager(db_manager=db)
```

```
active_accounts = auth_mgr.get_active_accounts()
```

```
active_acc_id = None
```

```
if active_accounts:
```

```
    print(f"\n🔑 Logged in Accounts ({len(active_accounts)}):")
```

```
    for acc in active_accounts:
```

```
        acc_id = acc["id"]
```

```
        email = acc["email"]
```

```
        creds = auth_mgr.get_credentials(account_id=acc_id)
```

```
        if creds:
```

```
            print(f" - [{acc_id}] {acc['name']} ({email}) -> Credentials Active & Valid ✅")
```

```
            if not active_acc_id:
```

```
                active_acc_id = acc_id
```

```
        else:
```

```
            print(f" - [{acc_id}] {acc['name']} ({email}) -> Credentials Invalid / Token Missing ❌")
```

```
else:
```

```
    print("\n🔒 No active accounts found. Starting Google Login Flow...")
```

```
    try:
```

```

account = auth_mgr.login()

active_acc_id = account["id"]

print(f"✅ Login Successful for: {account['email']}")

except Exception as e:

    print(f"❌ Login Failed or Cancelled: {e}")

print("\n✅ Nexora Core Initialization Complete!\n")

# 4. URL Resolver Analysis

url = "https://drive.google.com/drive/my-drive"

manager = URLManager()

result = manager.analyze(url)

if active_acc_id:

    resolv = DriveResolver()

    res = resolv.resolve(active_acc_id, url)

    print("🌀 URL Resolution Result:", res)

    # print(res[0][''])

    # scanner = DriveScanner()

    # list = scanner.list_shared_drives(acc_id)

    # print(list[0]['id'])

    # scan_info = scanner.scan_single_shared_drive(acc_id, list[0]['id'], list[0]['name'])

# 5. Launch Test UI

print("\n🖥️ Launching Sync Control UI Window...")

app = NexoraUI(db_manager=db, active_account_id=active_acc_id)

app.mainloop()

if __name__ == "__main__":

    main()

```